

# Trade Compute for Speed: Sampling Diffusion Models 2-4x Faster Without Losing Quality

Stanford's ParaDiGMS (NeurIPS 2023) reframes denoising as parallel Picard iteration

Diffusion models generate strikingly good images, and increasingly good robot actions, but they are slow. Producing a single sample can take up to 1000 sequential passes through the neural network, one denoising step feeding into the next. For anything interactive, that latency is the wall everyone hits first.

The field's standard answer is to take fewer steps. Fast samplers like DDIM and DPMSolver cut the 1000 steps down to a few dozen, and they work remarkably well, but the tradeoff is real: push the step count low enough and sample quality visibly degrades. We have largely accepted this as the deal, spending quality to buy speed.

This paper from Stanford asks a different question. What if, instead of trading quality for speed, we trade compute for speed? Keep every denoising step, but stop running them strictly one after another. The result is ParaDiGMS (Parallel Diffusion Generative Model Sampling), the first general method that spends extra parallel compute to sample a pretrained diffusion model faster, with no retraining and no measurable loss in quality.

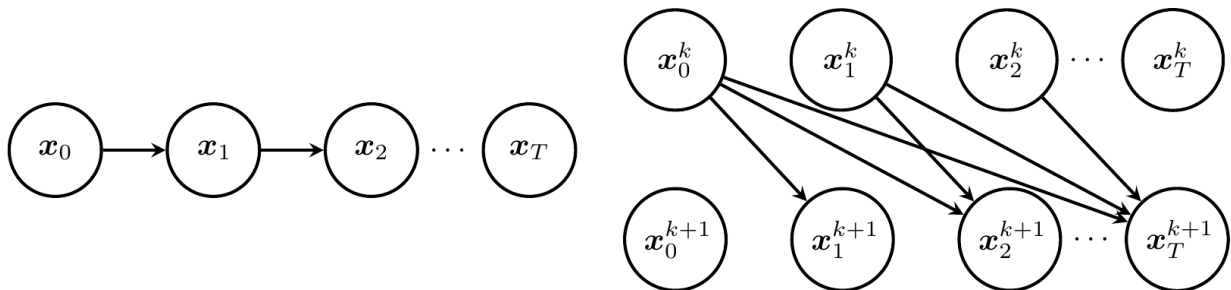


Figure 1. Two ways to compute a denoising trajectory. Left: sequential sampling evaluates one step at a time, so step  $t$  must finish before step  $t+1$  begins. Right: ParaDiGMS runs a Picard iteration that updates every timestep at once (superscript  $k$  is the iteration index), introducing skip dependencies so information can propagate down the whole chain in parallel.

Paper: <https://arxiv.org/abs/2305.16317>

Code: <https://github.com/AndyShih12/paradigms>

## Why single-sample latency is the hard part

It helps to be precise about what is slow. Diffusion sampling is sequential by construction: each denoising step reads the output of the previous one, so the 1000 steps form a chain you cannot obviously shortcut. You can trivially parallelize across samples, generating a whole batch of images at once, but that only raises throughput. It does nothing for the latency of the

single sample a user is waiting on. Making one sample appear faster in wall-clock time is the genuinely hard problem, and it is the one ParaDiGMS targets.

The authors' insight is that this sequential chain is really an ODE being solved by the simplest possible integrator, and there are better ways to solve an ODE than marching one step at a time.

## The method: denoising as a parallel fixed-point iteration

ParaDiGMS builds on Picard iteration, a classic technique for solving ordinary differential equations by fixed-point iteration. The idea is to write the value at each timestep as the starting point plus the accumulated drift along the path, then refine the whole trajectory at once. You begin with a rough guess for every timestep, evaluate the model's drift at all of them in parallel, and use those drifts to produce an updated guess for the entire trajectory. Repeat until the guess stops changing.

The reason this is a win is empirical but robust: the number of refinement iterations needed to converge turns out to be far smaller than the number of denoising steps, often by more than an order of magnitude. Each iteration touches all timesteps in parallel, so wall-clock time scales with the small iteration count rather than the large step count. To respect GPU memory, ParaDiGMS does not refine the full trajectory at once; it slides a batch window of fixed size  $p$  across the timeline, advancing the window as soon as the leading timesteps have converged. For stochastic (SDE) samplers, the noise is drawn up front so the resulting ODE stays well-behaved (Lipschitz).

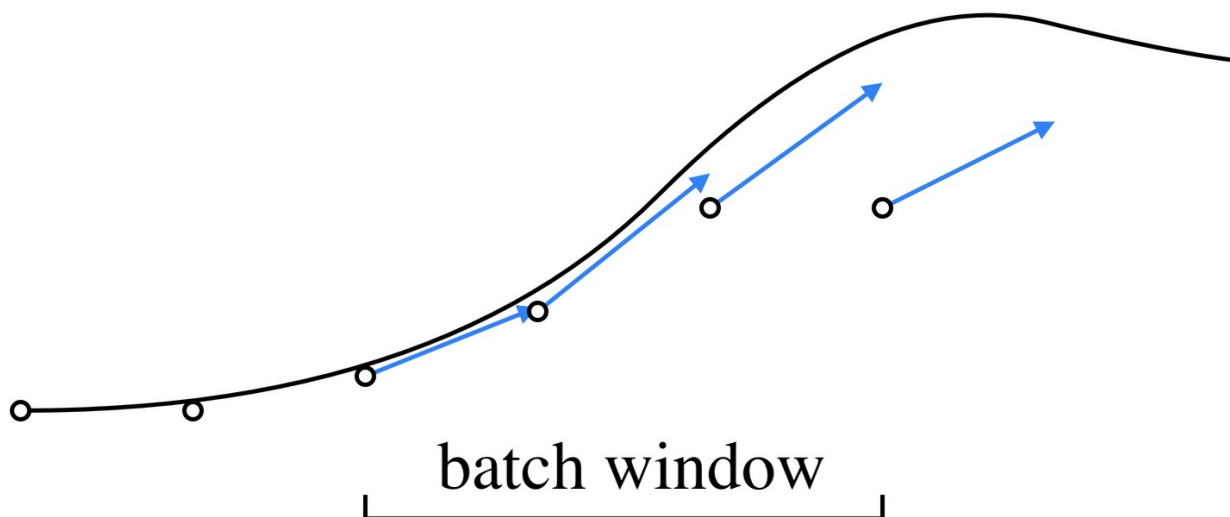


Figure 2. Inside the batch window. ParaDiGMS evaluates the drift (blue arrows) at every point in a window of size  $p$  simultaneously, uses the cumulative drift to update each point toward the true trajectory (black curve), and slides the window forward once the leading points have converged below the error tolerance.

Because it only changes how a fixed trajectory is computed, ParaDiGMS is orthogonal to the fast samplers people already use. Layer it on DDPM, DDIM, or DPMSolver and you get ParaDDPM, ParaDDIM, and ParaDPMSolver: the parallel speedup stacks on top of whatever step-count reduction those methods already provide. It is also compatible with classifier-free guidance.

## Results: a consistent 2-4x speedup, quality held flat

The headline is how consistent the gain is. Across two very different domains, robot control and image generation, and across every sampler tested, ParaDiGMS delivers a 2-4x reduction in sampling latency with no measurable drop in task reward, FID, or CLIP score. The deep reason it works: the number of parallel iterations to converge is up to 20x smaller than the number of sequential steps.

On robotics, the authors evaluate DiffusionPolicy models on three manipulation tasks, measuring task reward averaged over 200 evaluation episodes. Applying ParaDiGMS to the full 100-step DDPM policy (ParaDDPM) speeds up sampling by 3.4x to 3.9x while leaving reward essentially unchanged, bringing a 100-step policy down to about 0.2s per sample.

**Table 1. Robotics DiffusionPolicy (100-step DDPM, tolerance 0.1, window 20, single A40 GPU). Reward is averaged over 200 episodes; time is per episode. ParaDDPM holds reward flat while cutting latency 3.4-3.9x.**

| Task             | Reward (seq → Para) | Time (seq → Para) | Speedup |
|------------------|---------------------|-------------------|---------|
| Robosuite Square | 0.85 → 0.85         | 37.0s → 10.0s     | 3.7x    |
| PushT            | 0.81 → 0.83         | 32.3s → 8.33s     | 3.9x    |
| FrankaKitchen    | 0.85 → 0.84         | 112s → 33.3s      | 3.4x    |

On image generation the picture is the same. StableDiffusion-v2 generating 768x768 images from COCO captions drops from 50.0s to 14.6s per sample with ParaDDPM (3.4x) at an unchanged CLIP score of 32.1; stacking the parallel scheme on the already-fast DDIM and DPMSolver reaches 4.0x while CLIP holds at 31.7-31.9. On pixel-space LSUN Church, ParaDDPM matches full DDPM's FID (12.8 vs 12.9) at 3.9x.

**Table 2. Image generation on A100 GPUs. StableDiffusion-v2 quality is CLIP score (higher is better) over COCO2017 captions; LSUN Church is FID (lower is better) over 5000 samples. Quality is preserved across every configuration.**

| Model                        | Quality (seq → Para) | Time (seq → Para) | Speedup |
|------------------------------|----------------------|-------------------|---------|
| SD-v2, ParaDDPM (1000)       | CLIP 32.1 → 32.1     | 50.0s → 14.6s     | 3.4x    |
| SD-v2, ParaDDIM (200)        | CLIP 31.9 → 31.9     | 10.3s → 2.6s      | 4.0x    |
| SD-v2, ParaDPMSolver (200)   | CLIP 31.7 → 31.7     | 10.3s → 2.6s      | 4.0x    |
| LSUN Church, ParaDDPM (1000) | FID 12.8 → 12.9      | 24.0s → 6.1s      | 3.9x    |

What stands out across both tables is not any single number but the uniformity of the effect. A method that only helped latent-space image models, or only kicked in once quality was already saturated, would be a narrow trick. Seeing the same 2-4x hold from a 100-step robot policy to a 1000-step image model, and across FID, CLIP score, and task reward alike, is what makes the compute-for-speed trade look like a general property of diffusion sampling rather than a result tuned to one setting.

Because the speedup preserves quality, the samples themselves look no different from sequential output. The image below was generated with ParaDDPM running the full 1000-step StableDiffusion-v2 schedule over its 4x96x96 latent space, at 3.4x lower latency than the sequential baseline that produces an equivalent result.



Figure 3. A StableDiffusion-v2 sample from the prompt "a beautiful castle, matte painting," generated with ParaDDPM over the full 1000-step schedule on A100 GPUs. Parallel sampling produces the same quality output at 3.4x lower latency.

## The one knob: convergence tolerance

ParaDiGMS exposes a single, interpretable dial: the error tolerance of the fixed-point iteration. A tighter tolerance demands more iterations, giving higher fidelity but less speedup; a looser tolerance converges sooner and runs faster. The useful finding is that there is a wide comfortable regime where a fairly relaxed tolerance still leaves quality untouched, and only a genuinely loose setting begins to nudge quality down.

Table 3. Tolerance ablation on StableDiffusion-v2 with 200-step DDIM (CLIP over 1000 samples, A100 GPUs). Speedup grows steadily as tolerance loosens, and CLIP score stays flat until the tolerance becomes very large.

| Tolerance $\tau$ | Parallel iters | CLIP score | Speedup |
|------------------|----------------|------------|---------|
| seq              | 200            | 31.9       | -       |
| 5e-3             | 36             | 31.9       | 1.4x    |
| 1e-2             | 28             | 31.9       | 2.0x    |
| 5e-2             | 19             | 31.9       | 3.1x    |
| 1e-1             | 15             | 31.9       | 4.0x    |
| 5e-1             | 13             | 31.5       | 4.9x    |
| 1e-0             | 12             | 31.5       | 5.4x    |

One more control confirms the speedup is real rather than a disguised step-count reduction. On LSUN Church, ParaDDPM matches full DDPM's FID at nearly 4x, whereas simply shortening DDIM to 500 steps produces visibly worse images. The gains genuinely come from parallelism across all the steps, not from quietly taking fewer of them.

## Takeaway

ParaDiGMS opens a new axis for accelerating diffusion models. Rather than sacrificing quality by taking fewer steps, it spends parallel compute to run all the steps faster, cutting sampling latency by 2-4x with no quality loss, and it composes with the fast samplers already in wide use. The honest boundary is that this is a compute-for-speed trade: the extra parallel model evaluations are not free, so the win is largest when you have spare parallel hardware and care about latency rather than total FLOPs.

Looking ahead, that framing is also the interesting part. As GPUs keep getting better at large parallel batches, the wall-clock cost of sampling becomes bounded mainly by the small number of Picard iterations rather than the long chain of steps, pointing toward diffusion sampling that keeps getting faster as hardware scales, without ever paying for it in quality.